



SAPIENZA
UNIVERSITÀ DI ROMA

Foundations of Data Science

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente

Prof. Indro Spinelli

Appunti di

Alessio Cavedal

GitHub: <https://github.com/AlessioCavedal>

Anno Accademico 2026/2027

Indice

1	Data science e machine learning	1
1.1	Che cos'è la data science	1
1.2	Intelligenza artificiale e machine learning	2
1.3	I tre paradigmi dell'apprendimento	3
1.4	Supervised learning	3
1.5	Unsupervised learning	6
1.6	Reinforcement learning	7
2	I dati come vettori	9
2.1	I dati nel supervised learning	9
2.2	Tipi di dato ed encoding	10
2.3	Il dataset in termini formali	10
2.4	I punti come vettori	11
2.5	Spanning set e basi	12
2.6	La base standard e le basi ortonormali	13
2.7	Basi qualsiasi: encoding e calcolo dei pesi	13
2.8	Sottospazi e proiezioni	15

Capitolo 1

Data science e machine learning

Questo capitolo inquadra il corso: che cos'è la data science, perché al suo centro c'è il machine learning, e quali sono i tre grandi paradigmi dell'apprendimento automatico, ciascuno con i suoi esempi.

1.1 Che cos'è la data science

Definizione (data science). La **data science** è un campo interdisciplinare che estrae dai dati conoscenza e indicazioni concrete su cui agire (*actionable insights*). Comprende la raccolta, la pulizia, la gestione e l'analisi di dati strutturati e non strutturati, con tecniche che vengono dalla statistica, dalla matematica, dall'informatica e dalla conoscenza del dominio applicativo.

Un progetto di data science non è un'analisi isolata, ma un ciclo che si ripete (Figura 1.1). Le fasi sono cinque:

- *Business Understanding*: si definiscono gli obiettivi di business e li si traduce in obiettivi di machine learning;
- *Data Engineering*: si raccolgono e si verificano i dati, li si pulisce e li si visualizza, li si arricchisce con nuovi esempi (*data augmentation*) e li si standardizza;
- *Model Engineering*: si sceglie il modello di machine learning, lo si addestra e lo si valuta;
- *Deployment*: il modello va in produzione, e si verifica che gli utenti lo accettino e riescano a usarlo;
- *Monitoring*: se ne controlla l'efficacia nel tempo, lo si riaddestra quando serve e si raccolgono dati nuovi, che riportano all'inizio del ciclo.

Il machine learning compare in due punti chiave del ciclo: gli obiettivi vanno formulati come problemi di machine learning, e il cuore del lavoro tecnico è scegliere e addestrare un modello. Per questo il corso si concentra sul machine learning.

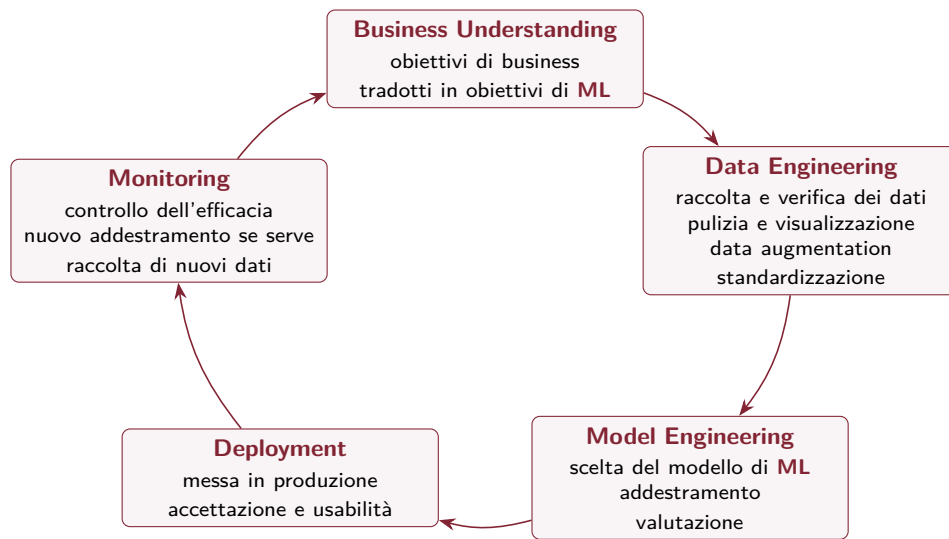


Figura 1.1. Il ciclo di vita di un progetto di data science. Il machine learning (ML) entra sia nella formulazione degli obiettivi sia nella scelta del modello.

1.2 Intelligenza artificiale e machine learning

L'**intelligenza artificiale** (*Artificial Intelligence*, AI) è il campo dell'informatica che si occupa di costruire agenti capaci di svolgere compiti che di solito richiedono l'intelligenza umana, come imparare, ragionare e risolvere problemi. Il **machine learning** (ML, apprendimento automatico) ne studia un aspetto preciso: l'apprendimento dall'esperienza. La definizione classica è di Tom Mitchell.

Definizione (machine learning). Il machine learning studia gli algoritmi che migliorano la propria *prestazione* P in un *compito* T grazie all'*esperienza* E . Un problema di apprendimento ben definito è dato dalla terna $\langle P, T, E \rangle$.

La definizione obbliga a dire con precisione che cosa si vuole imparare (T), come si misura il successo (P) e da quali dati si impara (E). La Tabella 1.1 riporta quattro esempi.

Il machine learning serve perché molti compiti, come riconoscere persone e oggetti in un'immagine o capire il parlato e il linguaggio, sono troppo complessi per scriverne le regole a mano. L'approccio del machine learning consiste allora nel costruire algoritmi che imparano da soli, a partire dai dati o dall'esperienza.

Machine learning e statistica. Machine learning e statistica hanno molto in comune: entrambi cercano regolarità (*pattern*) nei dati, usano analisi, probabilità e algebra lineare, e condividono molti algoritmi. Cambia l'enfasi. La statistica punta sull'inferenza: trarre conclusioni rigorose e interpretabili sul fenomeno che ha prodotto i dati. Il machine learning punta sulla previsione, sulla scalabilità e sull'autonomia del sistema. In breve: stessa matematica, obiettivi diversi.

Compito T	Prestazione P	Esperienza E
giocare a dama	percentuale di partite vinte contro un avversario qualsiasi	partite di allenamento giocate contro sé stesso
riconoscere parole scritte a mano	percentuale di parole classificate correttamente	database di immagini di parole scritte a mano, etichettate da persone
guidare su un'autostrada a quattro corsie con sensori visivi	distanza media percorsa prima di un errore, giudicato da una persona	sequenza di immagini e di comandi di sterzo registrati osservando un guidatore
classificare le email come spam o legittime	percentuale di email classificate correttamente	database di email, alcune etichettate a mano

Tabella 1.1. Quattro problemi di apprendimento descritti con la terna di Mitchell.

1.3 I tre paradigmi dell'apprendimento

Il machine learning si divide in tre paradigmi, a seconda dei dati da cui si impara:

- nel **supervised learning** (apprendimento supervisionato) si hanno esempi etichettati del comportamento corretto, cioè coppie formate da un input e dall'output desiderato;
- nell'**unsupervised learning** (apprendimento non supervisionato) non ci sono etichette, e si cercano nei dati strutture o regolarità «interessanti»;
- nel **reinforcement learning** (apprendimento per rinforzo) un sistema che apprende, l'**agente**, interagisce con il mondo e impara ad agire in modo razionale, massimizzando un segnale scalare di ricompensa (*reward*).

La Tabella 1.2 riassume che cosa riceve in ingresso e che cosa produce ciascun paradigma.

Paradigma	Input	Output
<i>Supervised</i>	esempi di input insieme all'output desiderato	un modello che predice l'output per un input nuovo
<i>Unsupervised</i>	esempi di dati, senza output	una rappresentazione della struttura dei dati
<i>Reinforcement</i>	una sequenza di interazioni con un ambiente	una <i>policy</i> , cioè una strategia che svolge il compito desiderato

Tabella 1.2. Input e output dei tre paradigmi dell'apprendimento.

1.4 Supervised learning

Nel supervised learning si vuole una funzione che porti dagli input agli output, e la si impara da esempi di coppie input-output. È un rovesciamento rispetto alla

programmazione tradizionale (Figura 1.2): lì il programmatore scrive il programma e il computer lo applica ai dati per produrre l'output; qui si forniscono al computer i dati insieme agli output attesi, ed è il computer a ricavarne il «programma», cioè il modello.

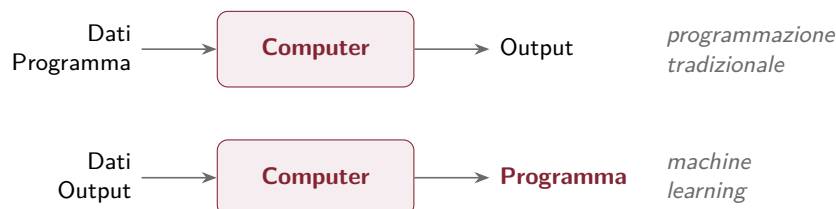


Figura 1.2. Nella programmazione tradizionale il programma è un ingresso; nel machine learning è il risultato, ricavato dai dati e dagli output attesi.

1.4.1 Dall'input reale all'output reale

Tutti gli esempi di supervised learning seguono lo stesso schema (Figura 1.3). L'input del mondo reale, che siano le caratteristiche di una casa, un testo o un'immagine, viene trasformato in un vettore di numeri, l'input del modello. Il modello produce un altro vettore, l'output del modello, che si interpreta di nuovo nel mondo reale: un prezzo, una classe, una frase. Un modello lavora soltanto su vettori di numeri: come si rappresentano i dati in questa forma è l'argomento del Capitolo 2.

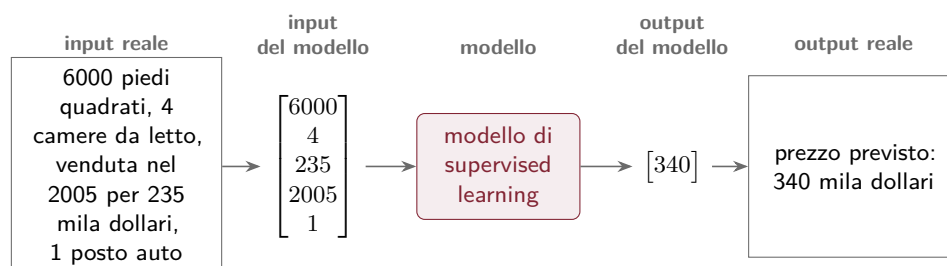


Figura 1.3. Lo schema comune agli esempi di supervised learning, nella stima del prezzo di una casa: l'input reale diventa un vettore, il modello produce un altro vettore, e questo si traduce nell'output reale.

1.4.2 Tipi di problema

In base all'output si distinguono:

- la **regressione**, in cui l'output è un numero reale: è **univariata** se l'output è uno solo, **multivariata** se sono più d'uno;
- la **classificazione**, in cui l'output è una classe scelta in un insieme discreto: è **binaria** se le classi sono due, **multiclasse** se sono più di due;
- la **classificazione binaria multivariata**, con molti output, ciascuno con due classi possibili: per esempio decidere, pixel per pixel, se un'immagine mostra un oggetto o lo sfondo.

Nei problemi di classificazione il modello di solito non restituisce direttamente la classe, ma un valore per ciascuna classe: nella classificazione di una recensione, per esempio, 0,02 per «positiva» e 0,98 per «negativa». Si sceglie la classe con il valore più alto.

La Tabella 1.3 raccoglie gli esempi della lezione, con il tipo di problema e l'architettura di rete neurale usata. Le architetture sono solo citate: per ora conta che compiti diversi richiedono modelli diversi.

Compito	Input → output	Tipo di problema	Architettura
regressione	caratteristiche di una casa → prezzo	regressione univariata	rete <i>fully connected</i>
<i>graph regression</i>	molecola → temperatura di congelamento e di ebollizione	regressione multivariata	<i>graph neural network</i>
classificazione di testi	recensione di un ristorante → positiva o negativa	classificazione binaria	<i>transformer</i>
genere musicale	brano audio → genere (classica, jazz, pop...)	classificazione multiclasse	rete ricorrente (RNN)
<i>image classification</i>	immagine → oggetto raffigurato (bicicletta, ape...)	classificazione multiclasse	rete convoluzionale
<i>image segmentation</i>	immagine → per ogni pixel, oggetto o sfondo	classificazione binaria multivariata	<i>encoder-decoder</i> convoluzionale
<i>depth estimation</i>	immagine → profondità di ogni pixel	regressione multivariata	<i>encoder-decoder</i> convoluzionale
<i>pose estimation</i>	immagine di una persona → posizione delle articolazioni	regressione multivariata	<i>encoder-decoder</i> convoluzionale
traduzione	testo in inglese → testo in francese	output strutturato	—
<i>image captioning</i>	immagine → frase che la descrive	output strutturato	—
generazione di immagini	descrizione testuale → immagine	output strutturato	—

Tabella 1.3. Esempi di supervised learning, con il tipo di problema e l'architettura di rete.

Gli ultimi tre esempi non rientrano nei tipi elencati sopra: l'output non è un numero né una classe, ma un oggetto strutturato (una frase, un'immagine), e le risposte valide possono essere molte.

1.4.3 Che cos'è un modello

Un modello di supervised learning è, in sostanza, un'equazione che lega l'input all'output. Nell'esempio della Figura 1.4 l'input è l'età di un bambino e l'output la sua altezza: per un bambino di 10 anni il modello prevede 139 cm. Addestrare il

modello significa cercare, in una *famiglia* di equazioni possibili, quella che si adatta bene ai dati di addestramento (*training data*), i punti del grafico.

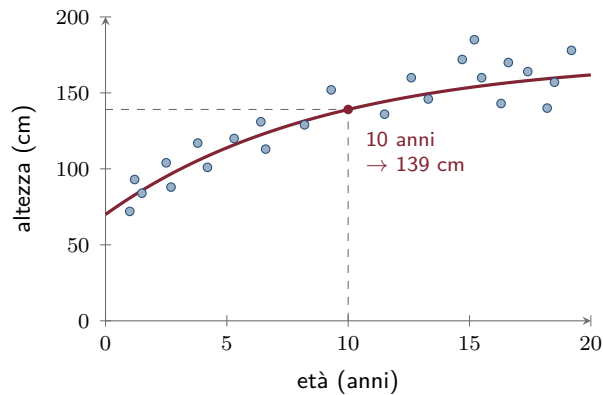


Figura 1.4. Un modello come equazione: la curva lega l'età all'altezza ed è scelta perché si adatta ai dati di addestramento (i punti). Per un bambino di 10 anni prevede 139 cm.

Le reti neurali profonde (*deep neural networks*) sono semplicemente una famiglia di equazioni molto flessibile, capace di descrivere relazioni complicatissime tra input e output. Adattarle ai dati è ciò che si chiama **deep learning**.

1.4.4 Che cosa hanno in comune gli esempi

Gli esempi della Tabella 1.3 hanno tre tratti comuni:

- la relazione tra input e output è molto complessa;
- a volte le risposte valide sono molte: una frase si traduce in più modi, e una descrizione corrisponde a infinite immagini;
- gli output, e talvolta gli input, obbediscono a delle regole: il linguaggio rispetta una grammatica, e anche le immagini naturali hanno le loro «regole», perché non ogni combinazione di pixel sembra una fotografia.

Da qui un'idea: imparare la «grammatica» dei dati da esempi senza etichetta, che si possono raccogliere in quantità enormi proprio perché nessuno deve etichettarli. Sapendo bene quali output sono plausibili, il compito supervisionato diventa più facile. È uno dei motivi per cui l'apprendimento non supervisionato è così importante.

1.5 Unsupervised learning

Nell'unsupervised learning si impara qualcosa su un dataset senza etichette. I compiti tipici sono:

- il **clustering**, cioè raggruppare i dati simili. Un esempio è DeepCluster (Caron et al., 2018), che raggruppa le immagini con caratteristiche visive simili senza sapere che cosa raffigurano;
- trovare gli **outlier**, i dati anomali;
- generare nuovi esempi, simili a quelli del dataset;

- ricostruire i dati mancanti.

Una classe importante è quella dei **modelli generativi**, capaci di creare nuovi esempi: da migliaia di foto di gatti, per esempio, imparano a produrre foto di gatti che non esistono. Ne sono un esempio le *generative adversarial networks* (GAN). Al loro interno, i **modelli generativi probabilistici** (*probabilistic generative models*, PGM) imparano esplicitamente una distribuzione di probabilità sui dati: tra questi i *variational autoencoder* (VAE), i *normalizing flow* e i *diffusion model*. I tre insieme sono uno dentro l'altro (Figura 1.5).

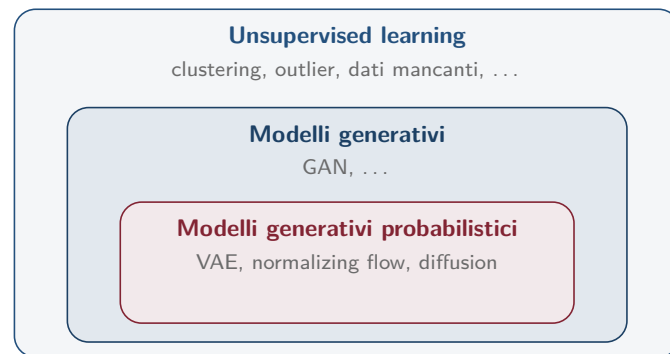


Figura 1.5. I modelli generativi sono una parte dell'unsupervised learning, e i modelli generativi probabilistici una parte dei modelli generativi.

Molti modelli generativi si basano sulle **variabili latenti** (Figura 1.6): si estraggono a caso pochi numeri da distribuzioni semplici, per esempio gaussiane, e un modello di deep learning li trasforma in un dato osservato ad alta dimensione, come un'immagine. L'idea è che dati complessi, fatti di moltissimi numeri, dipendano da pochi fattori nascosti; per generare un esempio nuovo basta estrarre nuovi valori delle variabili latenti.

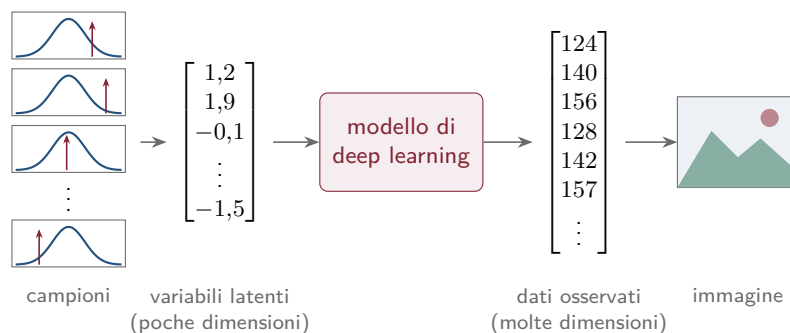


Figura 1.6. Variabili latenti: pochi valori estratti da distribuzioni semplici diventano, attraverso un modello di deep learning, un dato ad alta dimensione come un'immagine.

1.6 Reinforcement learning

Nel reinforcement learning ci sono un insieme di **stati**, un insieme di **azioni** e un insieme di **ricompense**. L'obiettivo è scegliere le azioni in modo da cambiare lo

stato e ricevere ricompense. A differenza degli altri due paradigmi non si riceve nessun dataset: l'agente deve esplorare l'ambiente e raccogliere i dati da sé, man mano che agisce (Figura 1.7).

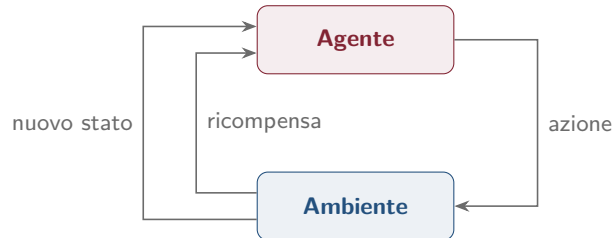


Figura 1.7. Il ciclo del reinforcement learning: l'agente sceglie un'azione, l'ambiente risponde con un nuovo stato e una ricompensa.

Esempio (gli scacchi). Gli stati sono le configurazioni valide della scacchiera e le azioni, in un dato momento, sono le mosse consentite. Le ricompense sono positive quando si mangia un pezzo dell'avversario e negative quando se ne perde uno.

Il problema è difficile per tre motivi:

- è **stocastico**: la stessa mossa ripetuta due volte può avere esiti diversi, perché l'avversario può rispondere in un altro modo, e anche le ricompense sono incerte (l'avversario può mangiare un pezzo oppure no);
- c'è il **temporal credit assignment problem** (attribuzione temporale del merito): quando arriva una ricompensa, non è chiaro se sia merito dell'ultima mossa o di buone scelte tattiche fatte molto prima;
- c'è il **exploration-exploitation trade-off** (compromesso tra esplorazione e sfruttamento): se si è trovata una buona apertura, conviene usare sempre quella o provarne altre, nella speranza di trovarne una migliore?

Capitolo 2

I dati come vettori

Nel capitolo precedente si è visto che un modello lavora su vettori di numeri. Questo capitolo guarda da vicino i dati: come sono fatti nel supervised learning, come li si converte in numeri e come li si descrive in termini formali. Poi tratta ogni dato come un vettore di uno spazio vettoriale, e da qui arriva alle nozioni di base, encoding e proiezione.

2.1 I dati nel supervised learning

Un esempio classico, introdotto dallo statistico Ronald Fisher e usato spesso come banco di prova, è il dataset **IRIS**: 150 fiori di iris, 50 per ciascuna di tre specie (*setosa*, *versicolor* e *virginica*), di cui sono misurate in centimetri lunghezza e larghezza del sepal e del petalo. Il compito è riconoscere la specie a partire dalle quattro misure.

In generale i dati del supervised learning sono misure di diversi aspetti di n oggetti, raccolte in una tabella con una riga per oggetto (Tabella 2.1). Le colonne sono di due tipi:

- il **target** è la variabile di output, l'obiettivo della previsione: qui la specie;
- le **feature** sono le proprietà che descrivono l'oggetto: qui le quattro misure.

Si assume che tra feature e target ci sia una relazione, e si vuole scoprire la regola predittiva che la descrive.

Feature $\mathbf{x}$				Target y
<i>Sepal.Length</i>	<i>Sepal.Width</i>	<i>Petal.Length</i>	<i>Petal.Width</i>	<i>Species</i>
4,3	3,0	1,1	0,1	setosa
5,0	3,3	1,4	0,2	setosa
7,7	3,8	6,7	2,2	virginica
5,5	2,5	4,0	1,3	versicolor

Tabella 2.1. Quattro righe del dataset IRIS: le misure in centimetri sono le feature, la specie è il target.

Le voci della colonna del target si chiamano **label** (etichette), e si distinguono due tipi di dati:

- i **dati etichettati** (*labeled data*), in cui il target è osservato;
- i **dati non etichettati** (*unlabeled data*), in cui il target è sconosciuto.

Il supervised learning impara dai primi; l'unsupervised learning (Sezione 1.5) lavora sui secondi.

2.2 Tipi di dato ed encoding

Feature e target possono essere di tipi diversi:

- **numerici**, con valori in $\mathbb{R}$;
- **interi**, con valori in $\mathbb{Z}$;
- **categorici**, con valori in un insieme finito di categorie $\{C_1, \dots, C_g\}$;
- **binari**, con valori in $\{0, 1\}$.

Il tipo del target decide il tipo di problema (Sezione 1.4): un target numerico dà un problema di regressione, un target categorico un problema di classificazione. IRIS, con la specie come target, è un problema di classificazione con tre classi.

Quasi tutti gli algoritmi di apprendimento lavorano solo con feature numeriche, salvo qualche eccezione. Le feature degli altri tipi vanno quindi convertite in numeri: bisogna scegliere, o costruire, un **encoding** (codifica) adatto.

Definizione (one-hot encoding). Con l'**encoding one-hot**, una variabile categorica con k categorie mutuamente esclusive passa da valore scalare a vettore binario di dimensione k , con un solo elemento uguale a 1, nella posizione della categoria, e 0 altrove. La categoria i -esima diventa

$$\mathbf{e}_i = (0, \dots, 0, \underbrace{1}_{\text{posizione } i}, 0, \dots, 0)^\top, \quad i \in \{1, \dots, k\}.$$

Esempio (le specie di IRIS). Con le specie nell'ordine setosa, versicolor, virginica, l'encoding one-hot è

$$\text{setosa} \mapsto (1, 0, 0)^\top, \quad \text{versicolor} \mapsto (0, 1, 0)^\top, \quad \text{virginica} \mapsto (0, 0, 1)^\top.$$

Se invece le categorie hanno un ordine naturale, come «basso», «medio» e «alto», si usa un encoding che lo rispecchi, per esempio una sequenza di interi: 1, 2, 3. Per categorie senza ordine, come le specie, gli interi sarebbero fuorvianti, perché introdurrebbero un ordine e delle distanze che non esistono (versicolor non sta «tra» setosa e virginica); i vettori one-hot invece sono tutti alla stessa distanza l'uno dall'altro.

2.3 Il dataset in termini formali

In termini formali, un dataset di n osservazioni è

$$D = ((\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})) \in (X \times Y)^n,$$

dove:

- X è lo **spazio degli input**, di solito $X \subseteq \mathbb{R}^p$ quando le feature sono p , tutte numeriche;
- Y è lo **spazio degli output**, o dei target.

Ogni osservazione, cioè ogni riga della tabella, è una coppia

$$(\mathbf{x}^{(i)}, y^{(i)}) \in X \times Y, \quad i = 1, \dots, n,$$

mentre la j -esima feature, cioè la j -esima colonna, è il vettore

$$\mathbf{x}_j = (x_j^{(1)}, \dots, x_j^{(n)})^\top \in \mathbb{R}^n.$$

L'indice in alto tra parentesi indica l'osservazione, quello in basso la feature: $x_j^{(i)}$ è il valore della feature j nell'osservazione i . Per IRIS $n = 150$ e $p = 4$, mentre lo spazio degli output è l'insieme delle tre specie, $Y = \{\text{setosa}, \text{versicolor}, \text{virginica}\}$.

2.3.1 Il processo che genera i dati

Si assume che i dati osservati siano prodotti da un processo sottostante, descritto da una distribuzione di probabilità P_{XY} definita su $X \times Y$. Le variabili aleatorie che seguono questa distribuzione si indicano con le minuscole $\mathbf{x}$ e y . La **distribuzione vera** P_{XY} di solito è sconosciuta: il supervised learning cerca di impararne la struttura, almeno in parte, quanto basta per prevedere y a partire da $\mathbf{x}$.

Definizione (ipotesi i.i.d.). Si assume che le osservazioni siano estratte da P_{XY} in modo **i.i.d.** (*independent and identically distributed*, indipendenti e identicamente distribuite): tutte provengono dalla stessa distribuzione e sono mutuamente indipendenti, cioè l' i -esima realizzazione non dipende dalle altre $n - 1$.

È un'ipotesi forte e non sempre realistica: in una serie temporale, per esempio, ogni osservazione dipende da quelle che la precedono. Esistono scenari più complessi, che ne fanno a meno, ma qui ci si limita al caso i.i.d.

2.4 I punti come vettori

Cambio di notazione. Da qui in avanti le slide, e con loro gli appunti, seguono la notazione del libro *Machine Learning Refined*:

- il dataset è formato da P punti $\mathbf{x}_1, \dots, \mathbf{x}_P \in \mathbb{R}^N$: P prende il posto di n , e N quello di p ;
- l'indice p scorre sui punti, l'indice n sulle dimensioni e sui vettori di una base;
- $\mathbf{x}_p$ è il p -esimo *punto* (una riga della tabella), non una colonna come $\mathbf{x}_j$ nella Sezione 2.3.

D'ora in poi si guarda solo agli input, ignorando il target. Si assume inoltre che il dataset sia **centrato** (*mean-centered*): a ogni punto si sottrae la media del dataset, dimensione per dimensione,

$$\bar{\mathbf{x}} = \frac{1}{P} \sum_{p=1}^P \mathbf{x}_p, \quad \mathbf{x}_p \leftarrow \mathbf{x}_p - \bar{\mathbf{x}},$$

così che i dati si dispongano attorno all'origine. È un'operazione semplice e del tutto reversibile: basta sommare di nuovo la media.

Ogni osservazione del dataset è un punto dello spazio degli input. Uno **spazio vettoriale** è un insieme di oggetti, i **vettori**, che si possono sommare tra loro e moltiplicare per numeri, gli **scalari**; $\mathbb{R}^N$ ne è l'esempio tipico. Un punto di $\mathbb{R}^N$ si può disegnare come un punto o come una freccia che parte dall'origine (Figura 2.1): sono due modi di vedere lo stesso oggetto, il primo più comodo per guardare un dataset, il secondo per ragionare su somme e multipli.

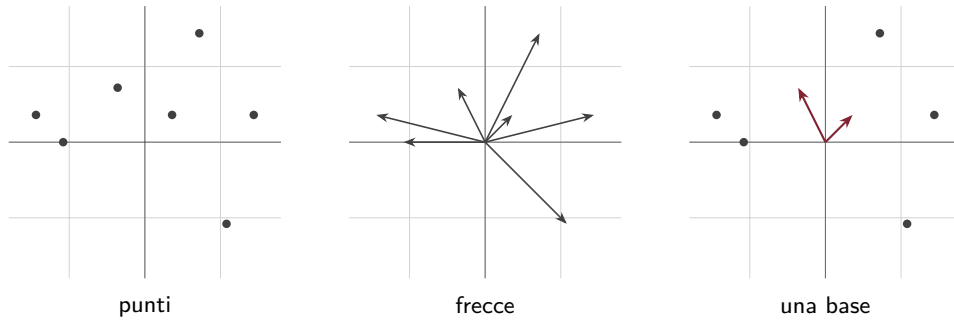


Figura 2.1. Lo stesso dataset in $\mathbb{R}^2$ visto come punti (a sinistra) e come frecce che partono dall'origine (al centro). A destra due soli vettori, che formano una base: ogni altro punto è una loro combinazione lineare.

2.5 Spanning set e basi

Definizione (spanning set). Uno **spanning set** (insieme di generatori) è un insieme di vettori che, combinati con somme e prodotti per scalari, producono *ogni* vettore dello spazio. Si dice che i vettori **generano** (*span*) lo spazio.

Per esempio, in $\mathbb{R}^2$ i vettori $\mathbf{e}_1 = (1, 0)^\top$ e $\mathbf{e}_2 = (0, 1)^\top$ generano il piano, perché ogni punto (x, y) si scrive come $x\mathbf{e}_1 + y\mathbf{e}_2$.

Definizione (base). Una **base** è uno spanning set minimale: il più piccolo insieme di vettori che genera ancora lo spazio. Nessun vettore deve essere ridondante, cioè i vettori devono essere **linearmente indipendenti**: nessuno si può ottenere come combinazione lineare degli altri.

In $\mathbb{R}^2$ la coppia $\{\mathbf{e}_1, \mathbf{e}_2\}$ è una base. Perché una base $\mathbf{c}_1, \dots, \mathbf{c}_N$ possa rappresentare perfettamente tutti i P punti del dataset, deve vivere nello stesso spazio $\mathbb{R}^N$

dei dati, e per ogni punto deve esistere un insieme di **pesi** con cui la combinazione lineare dei vettori della base restituisce il punto:

$$\sum_{n=1}^N \mathbf{c}_n w_{n,p} = \mathbf{x}_p, \quad p = 1, \dots, P. \quad (2.1)$$

Il peso $w_{n,p}$ dice quanto del vettore $\mathbf{c}_n$ serve per ricostruire il punto $\mathbf{x}_p$.

2.6 La base standard e le basi ortonormali

La base più semplice è la **base standard**: il vettore $\mathbf{c}_n$ ha tutti zeri tranne un 1 nella posizione n , cioè $\mathbf{c}_n = (0, \dots, 0, 1, 0, \dots, 0)^\top$. Le sue proprietà chiave sono due: vettori diversi sono ortogonali, e ogni vettore ha norma unitaria,

$$\mathbf{c}_n^\top \mathbf{c}_m = 0 \quad \text{per } n \neq m, \quad \|\mathbf{c}_n\|_2^2 = 1 \quad \text{per } n = 1, \dots, N.$$

Rappresentare un punto sulla base standard è banale: i pesi coincidono con il punto stesso, $\mathbf{w}_p = \mathbf{x}_p$, cioè ogni peso è il valore della coordinata corrispondente (Figura 2.2, a sinistra). I vettori one-hot della Sezione 2.2 sono proprio i vettori della base standard di $\mathbb{R}^k$.

Impilando i vettori della base come colonne di una matrice $\mathbf{C}$, le due proprietà si scrivono in forma compatta:

$$\mathbf{C}^\top \mathbf{C} = \mathbf{I}_{N \times N}.$$

L'elemento di posto (n, m) di $\mathbf{C}^\top \mathbf{C}$ è infatti il prodotto scalare $\mathbf{c}_n^\top \mathbf{c}_m$, che vale 1 sulla diagonale e 0 altrove. Un insieme di vettori con questa proprietà si dice **ortonormale**.

La base standard non è l'unica base ortonormale, e con una base ortonormale qualunque i pesi si trovano senza fatica. Sia $\mathbf{C}$ la matrice di una base ortonormale e $\mathbf{w}_p = (w_{1,p}, \dots, w_{N,p})^\top$ il vettore dei pesi di $\mathbf{x}_p$: la rappresentazione (2.1) diventa $\mathbf{C}\mathbf{w}_p = \mathbf{x}_p$. Moltiplicando a sinistra per $\mathbf{C}^\top$, e usando $\mathbf{C}^\top \mathbf{C} = \mathbf{I}$, si ottiene

$$\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p, \quad p = 1, \dots, P,$$

cioè $w_{n,p} = \mathbf{c}_n^\top \mathbf{x}_p$. Con una base ortonormale non c'è nessun sistema di equazioni da risolvere: il vettore dei pesi, chiamato **encoding** del punto, si ottiene con un semplice prodotto scalare tra ciascun vettore della base e il punto. Sostituendo l'encoding nella rappresentazione si ritrova il punto di partenza:

$$\mathbf{C}\mathbf{C}^\top \mathbf{x}_p = \mathbf{x}_p, \quad p = 1, \dots, P.$$

2.7 Basi qualsiasi: encoding e calcolo dei pesi

Se la base non è quella standard, il vettore dei pesi

$$\mathbf{w}_p = \begin{bmatrix} w_{1,p} \\ w_{2,p} \\ \vdots \\ w_{N,p} \end{bmatrix}$$

è diverso dal punto $\mathbf{x}_p$: è la sua nuova rappresentazione, il suo encoding, nel **transformed feature space** (spazio delle feature trasformato), lo spazio i cui assi coordinati sono i vettori della base.

Si prenda per esempio, in $N = 2$ dimensioni, la base

$$\mathbf{c}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{c}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

I due vettori sono linearmente indipendenti, perché puntano in direzioni diverse, quindi qualunque punto del piano si rappresenta come loro combinazione lineare:

$$w_{1,p}\mathbf{c}_1 + w_{2,p}\mathbf{c}_2 = \mathbf{x}_p,$$

con pesi $w_{1,p}$ e $w_{2,p}$ che sono unici per ogni punto $\mathbf{x}_p$. La Figura 2.2 confronta le due rappresentazioni dello stesso punto: con la base standard i pesi sono le coordinate del punto, con la base $\mathbf{c}_1, \mathbf{c}_2$ sono diversi, e nello spazio trasformato il punto ha per coordinate proprio quei pesi.

Esempio (calcolo dei pesi). Per il punto $\mathbf{x} = (5, 5)^\top$ (omettendo l'indice p) la condizione $w_1\mathbf{c}_1 + w_2\mathbf{c}_2 = \mathbf{x}$ è il sistema

$$\begin{cases} 2w_1 + w_2 = 5 \\ w_1 + 2w_2 = 5 \end{cases}$$

che ha soluzione $w_1 = w_2 = 5/3 \approx 1,667$. Allo stesso modo il punto $(-3, 0)^\top$ ha encoding $(-2, 1)^\top$: infatti $-2\mathbf{c}_1 + \mathbf{c}_2 = (-4 + 1, -2 + 2)^\top = (-3, 0)^\top$. La scorciatoia delle basi ortonormali qui non funziona: $\mathbf{C}^\top \mathbf{x} = (15, 15)^\top$ non è l'encoding, perché $\mathbf{c}_1$ e $\mathbf{c}_2$ non sono ortogonali ($\mathbf{c}_1^\top \mathbf{c}_2 = 4$).

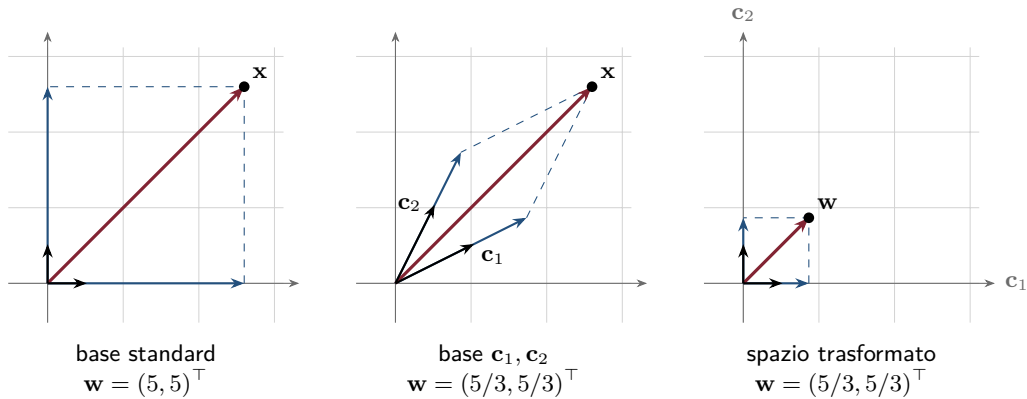


Figura 2.2. Il punto $\mathbf{x} = (5, 5)^\top$ sulla base standard (a sinistra) e sulla base $\mathbf{c}_1 = (2, 1)^\top$, $\mathbf{c}_2 = (1, 2)^\top$ (al centro): in nero i vettori della base, in blu i vettori scalati dai pesi, in rosso la loro somma, che è il punto. A destra l'encoding $\mathbf{w}$ nello spazio trasformato, i cui assi corrispondono a $\mathbf{c}_1$ e $\mathbf{c}_2$.

2.7.1 Trovare i pesi

In generale i pesi si trovano risolvendo un sistema di equazioni del primo ordine. Per ogni punto si cerca il vettore $\mathbf{w}_p$ che minimizza

$$h(\mathbf{w}_p) = \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2,$$

dove la matrice $\mathbf{C}$, di dimensione $N \times N$, ha per colonne i vettori della base:

$$\mathbf{C} = \begin{bmatrix} | & | & & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_N \\ | & | & & | \end{bmatrix}.$$

Annullando il gradiente di h , che vale $2\mathbf{C}^\top(\mathbf{C}\mathbf{w}_p - \mathbf{x}_p)$, si ottiene per ogni encoding un sistema lineare simmetrico (la matrice $\mathbf{C}^\top\mathbf{C}$ è simmetrica):

$$\mathbf{C}^\top\mathbf{C}\mathbf{w}_p = \mathbf{C}^\top\mathbf{x}_p, \quad p = 1, \dots, P.$$

Per trovare tutti i pesi insieme si può prendere la media dei costi dei singoli punti; il problema diventa

$$\min_{\mathbf{w}_1, \dots, \mathbf{w}_P} \frac{1}{P} \sum_{p=1}^P \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2.$$

2.8 Sottospazi e proiezioni

Ricapitolando, perché una base possa rappresentare perfettamente qualunque punto di uno spazio a N dimensioni servono due condizioni:

1. i vettori sono linearmente indipendenti, cioè puntano in direzioni diverse;
2. i vettori sono almeno N .

Che cosa succede se si rinuncia alla seconda, e i vettori sono solo $K < N$? Non si può più rappresentare perfettamente un insieme generico di P punti: in N dimensioni, K vettori generano al massimo un **sottospazio** di dimensione K . Per esempio, con $N = 3$:

- $K = 2$ vettori generano al più un piano, un iperpiano di $\mathbb{R}^3$;
- $K = 1$ vettore genera soltanto una retta.

In entrambi i casi non si raggiungono tutti i punti dello spazio, ma solo quelli di un sottospazio di dimensione più bassa. Un sottospazio passa sempre per l'origine: anche per questo conviene lavorare con dati centrati.

I pesi si trovano con equazioni della stessa forma di prima,

$$\mathbf{C}^\top\mathbf{C}\mathbf{w}_p = \mathbf{C}^\top\mathbf{x}_p, \quad p = 1, \dots, P,$$

con una sola differenza: ora $\mathbf{C}$ ha dimensione $N \times K$, perché contiene solo i K vettori della base, e quindi $\mathbf{w}_p \in \mathbb{R}^K$:

$$\mathbf{C} = \begin{bmatrix} | & | & & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_K \\ | & | & & | \end{bmatrix}.$$

Questo procedimento si chiama **proiezione**, perché equivale a «far cadere» il punto perpendicolarmente sul sottospazio generato dai K vettori (Figura 2.3). Il vettore dei pesi $\mathbf{w}_p$ è l'encoding del punto nel sottospazio: K numeri invece di N . Il passo di **decoding** ricostruisce il punto combinando l'encoding con i K vettori della base: il risultato $\mathbf{C}\mathbf{w}_p$ è il **punto proiettato**, la migliore approssimazione del punto originale all'interno del sottospazio (Figura 2.4).

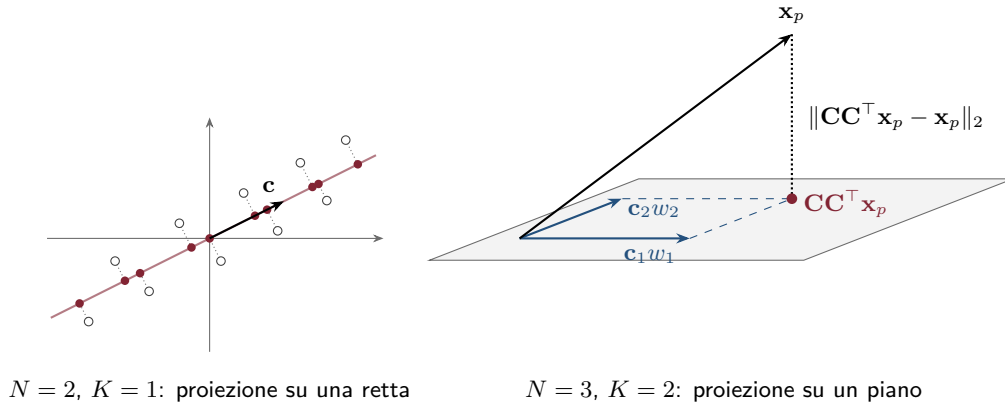


Figura 2.3. A sinistra, punti di $\mathbb{R}^2$ (cerchi vuoti) proiettati sulla retta generata da un solo vettore $\mathbf{c}$: i punti pieni sono le proiezioni. A destra, un punto $\mathbf{x}_p$ di $\mathbb{R}^3$ proiettato sul piano generato da due vettori ortonormali $\mathbf{c}_1$ e $\mathbf{c}_2$: il punto proiettato è $\mathbf{C}\mathbf{C}^\top \mathbf{x}_p = \mathbf{c}_1 w_1 + \mathbf{c}_2 w_2$, e il segmento punteggiato è l'errore di approssimazione.

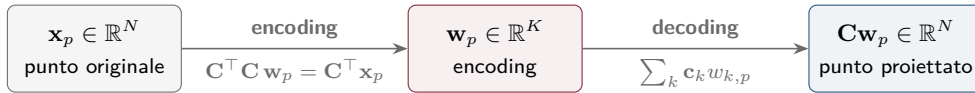


Figura 2.4. Encoding e decoding: il punto originale, fatto di N numeri, diventa un encoding di K numeri, da cui il decoding ricostruisce il punto proiettato, di nuovo in $\mathbb{R}^N$.

Se la base di K elementi è ortonormale, la formula dell'encoding si riduce di nuovo a

$$\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p, \quad p = 1, \dots, P,$$

e il decoding dà

$$\mathbf{C}\mathbf{C}^\top \mathbf{x}_p \approx \mathbf{x}_p, \quad p = 1, \dots, P.$$

Al posto dell'uguaglianza della Sezione 2.6 ora c'è un'approssimazione: il punto proiettato coincide con $\mathbf{x}_p$ solo se $\mathbf{x}_p$ sta già nel sottospazio. L'errore commesso è la distanza tra il punto e la sua proiezione, $\|\mathbf{C}\mathbf{C}^\top \mathbf{x}_p - \mathbf{x}_p\|_2$.